Lesson 13: A Hit That Reacts

Unity

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Create a small Explosion prefab: a GameObject with a SpriteRenderer (Sprite = explosion_placeholder.png) and a script that calls Destroy(gameObject, 0.3f) in Start() - it cleans itself up automatically, no help needed from anything else.
3. Import enemy_invader_white.png (this lesson's Assets folder) into your project - it's the same enemy silhouette as Lesson 4's sprite, pre-recolored solid white, used below for the hit flash.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

4. Hit detection stays exactly where it's always been since Lesson 4 - on Bullet.cs's own OnTriggerEnter2D, checking other.CompareTag("Enemy"). Nothing about WHERE the hit is detected changes this week.
5. In Enemy.cs, add public GameObject explosionPrefab, public SpriteRenderer spriteRenderer, and public Sprite flashSprite fields, and assign them in the Inspector (flashSprite = enemy_invader_white.png). Add using System.Collections; to Enemy.cs's using block if it isn't already there - Coroutines need it.
6. In Enemy.cs, add a new Coroutine - public IEnumerator HandleHit() (code below) - Unity's first Coroutine this course, so the Enemy can pause its OWN removal partway through without freezing the rest of the game. Inside it: Instantiate the explosion immediately, swap spriteRenderer.sprite to flashSprite, then yield return new WaitForSeconds(0.1f), then Destroy(gameObject) - that's the only line actually delayed. A color tint (spriteRenderer.color) won't work here: it multiplies the sprite's existing color, and multiplying any color by white leaves it completely unchanged - a genuinely white sprite is the only way to get a visible white flash on colored source art, the same reason Lesson 2's Recolor swaps sprites instead of tinting them.
7. Back in Bullet.cs's OnTriggerEnter2D: keep hitSound and GameManager.RegisterKill() running immediately and unchanged - score/sound/wave/win-check must still happen at exactly the same instant as last week. Right after those, instead of calling Destroy(other.gameObject) directly, grab the Enemy component and start ITS Coroutine instead: Enemy enemy = other.GetComponent<Enemy>(); enemy.StartCoroutine(enemy.HandleHit());. A Coroutine always runs on whichever

MonoBehaviour you call StartCoroutine() from - starting it on enemy, not on Bullet itself, is what lets the Enemy keep pausing and flashing even after the Bullet that hit it is already gone.

8. Add one more line right before that: other.enabled = false; (disabling the Collider2D that was passed in). The Enemy no longer disappears the instant it's hit - it stays on screen, flashing, for 0.1 seconds - so without this, a second overlapping bullet could trigger OnTriggerEnter2D against the very same, already-hit enemy and double-register a kill.
9. Bullet.cs still destroys itself (Destroy(gameObject)) immediately, exactly as it always has - only the Enemy's own removal is delayed now, and it's the Enemy's own Coroutine doing the delaying, not Bullet's.
10. Also notice the hit sound now plays via AudioSource.PlayClipAtPoint(hitSound, Camera.main.transform.position) - at the camera, not the enemy's own position - matching the same pattern your other round-ending sounds already use, not Lesson 4's original (and, as of this lesson, corrected) transform.position placement.
11. Press Play and play through a full round - each hit should flash the enemy white briefly, and a small burst should appear at that spot for a moment.

Highlighted lines are new this week; everything else is code you already have.

```
// Bullet.cs's OnTriggerEnter2D (revised):
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        other.enabled = false;
        Enemy enemy = other.GetComponent<Enemy>();
        AudioSource.PlayClipAtPoint(hitSound,
Camera.main.transform.position);
        GameManager.RegisterKill();
        enemy.StartCoroutine(enemy.HandleHit());
        Destroy(gameObject);
    }
}

// Enemy.cs (new fields and Coroutine):
public GameObject explosionPrefab;
public SpriteRenderer spriteRenderer;
public Sprite flashSprite;

public IEnumerator HandleHit() {
    Instantiate(explosionPrefab, transform.position, Quaternion.identity);
    spriteRenderer.sprite = flashSprite;
    yield return new WaitForSeconds(0.1f);
    Destroy(gameObject);
}
```

Alternate Path: Visual Scripting

The same flash-then-explode idea works in a Script Graph, using a Wait For Seconds node in place of a C# Coroutine - split across Bullet's graph (which still detects the hit) and a new graph on Enemy (which handles its own delayed removal).

12. On Bullet's existing hit-detection graph, right after the existing RegisterKill and sound nodes (unchanged), add a Set Variable node (Enabled = false) targeting the Enemy's Collider2D, then a node that triggers a Custom Event named HandleHit on the Enemy object specifically - not on Bullet's own graph.

13. On the Enemy's own graph, add a Custom Event node named HandleHit that chains: Instantiate (explosion prefab) -> Set Sprite (flashSprite, not Color, which would multiply against the enemy's existing color and produce no visible change) -> Wait For Seconds (0.1) -> Destroy (self). Triggering this event on the Enemy, rather than running the wait on Bullet's own graph, is what lets the Enemy keep flashing after Bullet is already gone.

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - yield only pauses the Enemy's OWN Coroutine, nothing else in the game pauses.
- Confirm explosion prefabs never pile up in the Hierarchy - each one destroys itself via Destroy(gameObject, 0.3f), with no help from the Enemy.
- Shoot two bullets at the same enemy in quick succession and confirm score only goes up once - other.enabled = false disabling the Collider2D the instant it's hit is what prevents the second bullet from also registering a kill.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: instead of one flat instant swap, alternate spriteRenderer.sprite between flashSprite and the enemy's normal sprite 2-3 times across the 0.1 seconds (with tiny waits between) for a flicker instead of a single snap.
- Visual: scale the explosion prefab up slightly over its lifetime with a small script using Time.deltaTime, so it feels like it's expanding.
- Structure: notice the explosion prefab knows nothing about score, lives, or waves - it's a fully self-contained object, the same discipline as this course's bullets and enemies.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm Bullet.cs is actually calling enemy.StartCoroutine(enemy.HandleHit()) rather than destroying the enemy directly, and that HandleHit() itself is a real Coroutine (IEnumerator, not void).
- Enemy disappears but never visibly turns white: confirm flashSprite is actually assigned in the Inspector to enemy_invader_white.png, not left empty or still pointing at the normal sprite.
- Explosion never appears: confirm explosionPrefab is actually assigned in the Inspector, not left empty.
- Score increases by 20 instead of 10 on some hits: confirm other.enabled = false runs before enemy.StartCoroutine(...) in Bullet.cs's OnTriggerEnter2D, not after - a second overlapping bullet can otherwise trigger the same still-alive, still-flashing enemy a second time.
- Hit sound is missing or barely audible: confirm hitSound is actually assigned in the Inspector, and that the call uses Camera.main.transform.position, not transform.position - PlayClipAtPoint() always creates a 3D audio source with real distance rolloff, so a sound played at the enemy's own position (far from the camera/listener) can end up almost inaudible even though it's technically playing.

Want More?

Student Resource List > Unity section, or learn.unity.com